讓 AI Agent 記住你

從零打造仿生長期記憶系統的設計決策

吳勝繙 Chris Wu｜數位發展部

HWDC 2026 ・ 2026-10-15 ・ Session 2 ・ 中階 ・ 30 分鐘

記憶，決定 Agent

在對的時刻，知道對的事。

你的 Agent，其實每次都不記得你

四個工作場景，四個記憶帶來的差距

任務開始：不用重新解釋流程和規則

下週繼續：直接接上，不用重新交代背景

多條工作線：各自獨立，互不干擾

越用越好：偏好和習慣越來越精準

記憶讓 Agent 有歷史

一個長期開發專案的真實紀錄

跨週追蹤設計決策，下次直接接上

Anti-pattern 記住踩過的坑，不重蹈覆轍

SOP 沉澱後，每次提案品質持續進步

記憶不只是查詢，是 Agent 認識你的方式

RAG 給你文件，Agent 需要的是感知先驗

三個 RAG 覆蓋不到的缺口

語意相似 ≠ 相關：踩過的坑影響最大卻語意距離遠

靜態無生命週期：寫入後無過期晉升衰退管理

無認知閉環：哪些知識被用哪些沒用 RAG 無感

研究與產業都指向同一件事

RAG 知識不累積，這是架構限制，不是設計選擇

Karpathy（GitHub Gist, Apr 2026）：RAG 每次重新拼答案，知識不累積

Dream-RSI（arXiv:2609.14858）：壓縮成功路徑，失敗記憶不能同等對待

Claude Code：業界開始內建做夢機制，認可記憶需主動管理

arXiv:2602.02007 · arXiv:2608.22215

決策一：依來源差異化遺忘

神經科學靈感

海馬體 dual-store：

快速暫存（海馬體）vs 慢速長期（新皮質）

不同性質記憶有不同轉移速度

設計結論

來源決定信任
信任決定壽命

記憶從寫入那刻就知道該信多少

DECISION 01 / 04

記憶從寫入的那一刻，就知道該信多少

這不是儲存，是有判斷力的吸收

來源衰減不同：外部資料快衰，自己說的慢衰

Write gate 在寫入端攔截低信任來源

Recall 標記讓 LLM 自行決定是否採納

同樣一筆資訊，有無信任分級的差距

沒有信任分級

外部資訊和自產知識一視同仁

過時資料繼續影響推理

LLM 無從判斷該信哪一筆

有信任分級

外部來源快速衰退，自產知識保留

Write gate 擋下低信任來源

LLM 看到 recall 標記，自己裁決

DECISION 01 / 04

生產數據：news-events bank 265 筆 stale，外部來源佔多數

決策二：從錯誤中強化記憶

神經科學靈感

杏仁核恐懼記憶：

情緒性錯誤觸發更強烈記憶固化

帶來痛苦的錯誤比中性記憶更難遺忘

設計結論

踩過的坑，
永遠比新知識更重要

成功可以忘，失敗不行

DECISION 02 / 04

成功可以忘，失敗不行

踩過的坑，永遠比新知識更重要

Anti_pattern 直接進耐久記憶，跳過壓縮流程

低使用率不代表不重要：坑只踩一次但那次關鍵

Dream-RSI 互補：壓縮成功路徑讓探索變差

這個設計的已知問題

反例永遠留著，長期會膨脹——解法不是刪除，是壓縮

問題：anti_pattern 無上限，長期記憶庫膨脹

解法：OKF 蒸餾——同主題反例壓縮為一筆高密度知識

原則：失敗記憶不消失，多筆變成一筆精煉知識

決策三：以使用價值驅動晉升

神經科學靈感

Hebbian plasticity：

反覆激活 → 突觸連結強化

不活躍 → 連結弱化消退

設計結論

越被用到的，
越應該留下來

使用次數是記憶系統的市場機制

DECISION 03 / 04

為什麼不用時間，用使用頻率

舊不等於沒用，新不等於有用

時間衰退：會清掉關鍵但只被用過一次的記憶

使用頻率才是真實價值，集中在偏好和 SOP

正反饋迴圈到 5.5x，LOG_2 阻尼壓回 2x

決策四：以新穎性判斷避免重複儲存

神經科學靈感

海馬體 pattern separation：

持續比對輸入與記憶

熟悉刺激降低反應，不重新編碼

設計結論

垃圾進，垃圾出

寫入前先判斷，比事後清理便宜

在記憶系統比在資料庫更嚴重

DECISION 04 / 04

寫入前先判斷——記憶庫品質靠入口管控

垃圾進，垃圾出——在記憶系統比在資料庫更嚴重

Retain 前計算新穎性分數

四種結果：全新 / 強化 / 重複（攔截）/ 衝突（標記）

衝突第一時間標記，讓 LLM 在 recall 時看到信號

新穎性判斷的設計邊界

系統提供 hint，Agent 做決定

跨 bank 去重是被動的：系統給 hint，Agent 決定

衝突不自動解決：可能是情境不同而非真矛盾

DECISION 04 / 04

為什麼自建，而不是現成方案

不是現成方案不好，是沒有一個同時解決這四件事

Mem0：沒有 write gate，所有來源等權（官方明載）

Zep / Letta：都沒有使用頻率驅動的生命週期

代價：自建意味著架構演進和維護成本要自己承擔

記憶系統是 Agent 架構的一層，不是 RAG 的升級

確定性工具、小模型、大模型——各司其職

確定性工具負責生命週期，不需要 LLM

小模型做局部語意判斷，便宜且快

大模型只做最終推理，有上下文才裁決

差別不在模型有多強，

在於它是否在協作的過程中

真的幫助到你。